

production and usage, in spite of their well-known inefficiency compared to ASIC and FPGA-based solutions.

Given this mismatch between the extreme area and power constraints of emerging applications and the relative inefficiency of general-purpose microprocessors and microcontrollers compared to their ASIC counterparts, there exists a considerable opportunity to make microprocessor-based solutions for these applications much more area- and power-efficient.

The bespoke processor design methodology relies on gate-level symbolic simulation to identify gates in the microprocessor IP that cannot be toggled by the application, irrespective of the application inputs, and automatically eliminates them from the design to produce a significantly smaller and lower power design with the same performance. In many cases, reduction in the number of gates also introduces timing slack, which can be exploited to improve performance or further reduce power and area. Since the original design is pruned at the granularity of gates, the resulting methodology is much more effective than any approach that relies on coarse-grained application-specific customisation.

This methodology can be used either by IP licensors or IP licensees to produce bespoke designs for the application of interest. Its simple extensions can be used to generate bespoke processors that can support multiple applications or different degrees of in-field software programmability, debuggability and updates.

In the world of ultra-low-power applications, power is everything. There are three basic types of systems depending on the sources of power:

1. Systems powered directly by energy harvesting (for example, a solar cell)
 2. Systems powered by a battery, which, in turn, is charged via energy harvesting
 3. Systems that just have a battery
- It's the size of energy harvesting and/or energy storage (battery) components that ultimately determines the form factor, size and weight of an ultra-low-power device. Peak power and energy consumption impact har-

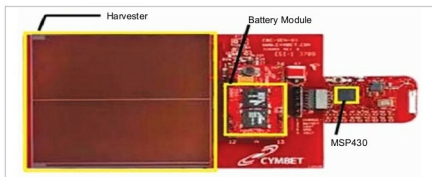


Fig. 6: Harvester with battery

vester and battery size calculations in different ways depending on the type of device.

Current approaches for determining peak power and energy

There are three basic approaches for determining the peak power and energy requirements of an ultra-low-power processor for a given application. For a very conservative upper bound, look at the datasheets for the processor, which tell the peak power that can be consumed by the hardware. Run a stress mark—an application that attempts to activate the hardware in a way that maximises peak power or energy. A stress mark may be less conservative than a design specification, since it may not be possible for an application to exercise all parts of the hardware at once. Perform application profiling on the processor by measuring power consumption while running the target application on the hardware.

Things tend not to work out well if a processor tries to operate outside of the peak power and energy bounds available to it. So the best practice is to add a guard band (power buffer) to any profiling-based results. A typical guard banding factor might be 33 per cent. In other words, take the profiling results, add 33 per cent and provision for that.

Input-independent peak power and energy profiling

The core idea is to run a symbolic simulation of an application binary on the gate-level netlist of a processor. Most ultra-low-power systems tend to have simple processors and applica-

tions, making simulation feasible. For example, even the most complex benchmark may complete full simulation in two hours.

During simulation, a special X value is propagated for all signal values that can't be constrained based on the application. Start with all gates and memory locations not explicitly loaded with the binary set to X. Any input values during simulation are also set to X.

As simulation progresses, the simulator dynamically constructs an execution tree describing all possible execution paths through the application. If an X symbol propagates to the inputs of the program counter (PC) during simulation, indicating an input-dependent control sequence, a branch is created in the execution tree. Normally, the simulator pushes the state corresponding to one execution path onto a stack for later analysis and continues down the other path.

At the end of this process, you know the activity of each gate at each point in the execution tree. A gate that is not marked as toggled (0 to 1, or 1 to 0) at a particular location in the tree can never be toggled at that location in the application. You can use this knowledge encoded in the execution tree to generate peak power requirements as follows:

1. Concatenate all of the execution paths into a single execution trace.
2. Assign all of the Xs in the trace such that power for each cycle is maximised. Power is maximised when a gate toggles, but a transition requires two cycles—one to prepare and the other to make the transition. Since you don't know the best way to align transitions with cycles, two separate